Do Protótipo ao Produto Seguro: Avaliação de Segurança do NexaSense

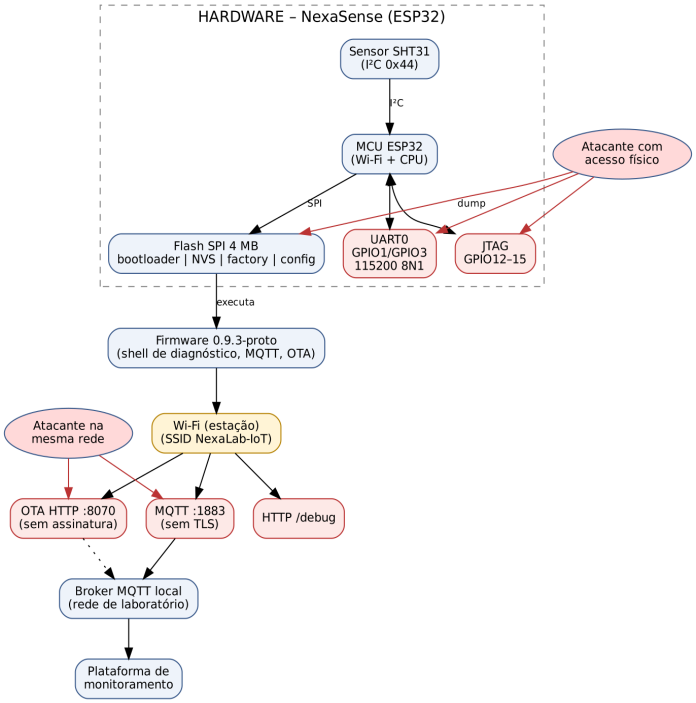
IoT Security Analyst · NexaHome Technologies · Hardware Hacking IoT · 01/11/2026
Autor: Matheus Ferreira Vieira · RA 99693 · Repositório: github.com/12matheusf/Nexahome-iot-security · Vídeo: youtu.be/JNwaTdVuQTU

Aviso de escopo e transparência. Avaliação em ambiente local, no notebook do analista (Windows, Npcap 1.90) e na sua VM Kali (QEMU, binwalk, strings). Nenhum firmware ou placa foi fornecido para a atividade, então há duas camadas de evidência: (1) cenário didático NexaSense, com firmware e dispositivo simulados (credenciais fictícias, log UART reconstruído), usado para demonstrar a metodologia, a captura de tráfego e as mitigações; (2) firmware real e público (Tasmota32) executado em ESP32 emulado (QEMU), com análise de flash e UART verdadeiras. Nenhuma placa física foi usada neste relatório. Nenhum sistema de terceiros foi testado.

1. Dispositivo analisado

O NexaSense é um sensor de temperatura e umidade baseado em ESP32 (flash de 4 MB, sensor SHT31 via I²C) que envia telemetria por Wi-Fi usando MQTT, recebe atualizações OTA e mantém recursos de depuração (UART, JTAG, página HTTP /debug) herdados do protótipo. Firmware 0.9.3-proto, ESP-IDF v4.4.6.

2. Arquitetura e componentes



Componente	Função	Observação de segurança
MCU ESP32	Execução do firmware, Wi-Fi	JTAG não desabilitado (eFuse)
Flash SPI 4 MB	bootloader, NVS, app, config	Sem Flash Encryption; entropia 0,007
Sensor SHT31 (I²C 0x44)	Temperatura/umidade	Barramento interno, risco baixo
UART0 / JTAG	Console e depuração	Acessíveis e sem proteção
Wi-Fi + MQTT/OTA/HTTP	Comunicação	Sem TLS, OTA sem assinatura

3. Mapa da superfície de ataque

Superfície	Vetor	Atacante	Achados
Física	UART, JTAG, dump da flash SPI	Acesso físico à placa	F01, F05, F06
Firmware	Segredos e configuração embarcados, ausência de Secure Boot	Acesso à imagem (flash ou OTA)	F01, F03, F05, F07

Superfície	Vetor	Atacante	Achados
Rede	MQTT 1883, OTA HTTP, /debug	Mesma rede / MITM	F02, F03, F04, F07

4. Metodologia

Planejamento e organização dos testes segundo o OWASP IoT Security Testing Guide; ameaças selecionadas com o MITRE EMB3D; comparação de capacidades com NIST IR 8259A; requisitos de hardening da ETSI EN 303 645; controles do chip conforme ESP-IDF Security. Fluxo por achado: evidência → fragilidade → ameaça → impacto → risco → mitigação. Risco = matriz 3×3 impacto × probabilidade. Interface acessível não foi tratada como vulnerabilidade automática: avaliou-se contexto, proteções existentes e impacto.

5. Análise do firmware

Original preservado como somente leitura; SHA-256 acaa7151e2a60c3117a7b15ce7d72893127f7fe94fd9bf0a7723f985803cbef4 (4.194.304 bytes); cópia de trabalho com hash idêntico. Estrutura: bootloader (0x1000), tabela de partições (0x8000), NVS, otadata, phy_init, factory (0x20000) e config (0x1A0000). Entropia global 0,007 bits/byte indica imagem não cifrada. Strings revelam senha de Wi-Fi, token de API, senha MQTT, admin:admin, endpoints (mqtt://...:1883, http://...:8070), 'signature verification SKIPPED', 'TLS cert verification disabled', 'secure boot: disabled', 'flash encryption: disabled' e 'JTAG_DISABLE=0'. Relevância: qualquer pessoa com a imagem (dump ou OTA interceptada) obtém credenciais e sabe que o dispositivo aceitaria firmware não assinado.

6. Análise das interfaces físicas

Interface	O que permite	Proteção	Conclusão
UART0 (GPIO1/3, 115200)	Log de boot (imprime senha MQTT) e shell	Nenhuma	Exposição relevante com acesso físico (F06)
JTAG (GPIO12–15)	Depuração/leitura de memória	eFuse não queimado	Exposição relevante (F05)
SPI flash	Dump completo	Sem Flash Encryption	Exposição relevante (F01, F05)
I²C (SHT31)	Interação com sensor	Barramento interno	Baixa relevância

Raciocínio: a UART não é “insegura por existir”; é relevante porque fornece segredos que dão acesso ao broker (credencial compartilhada) e não há proteção compensatória.

7. Análise das comunicações e demonstração controlada

No laboratório, um broker MQTT local e um dispositivo simulado foram executados com Scapy capturando a interface de loopback. Demonstração: (1) credenciais MQTT extraídas do firmware por regex; (2) o dispositivo as usa; (3) o pcap mostra usuário e senha em claro no pacote CONNECT; (4) um assinante sem credenciais leu 4/4 mensagens de telemetria em nexahome/#. Protocolo: MQTT 3.1.1/TCP 1883, sem TLS; identificadores (NEXASENSE-0001) expostos. Identificar vs. avaliar: saber que o protocolo é MQTT não diz se é seguro; o mesmo MQTT sobre TLS com certificado por dispositivo e ACL seria adequado. Reprodução: python3 lab/run_network_lab.py. Limitação: tráfego simulado; OTA e /debug avaliados só estaticamente.

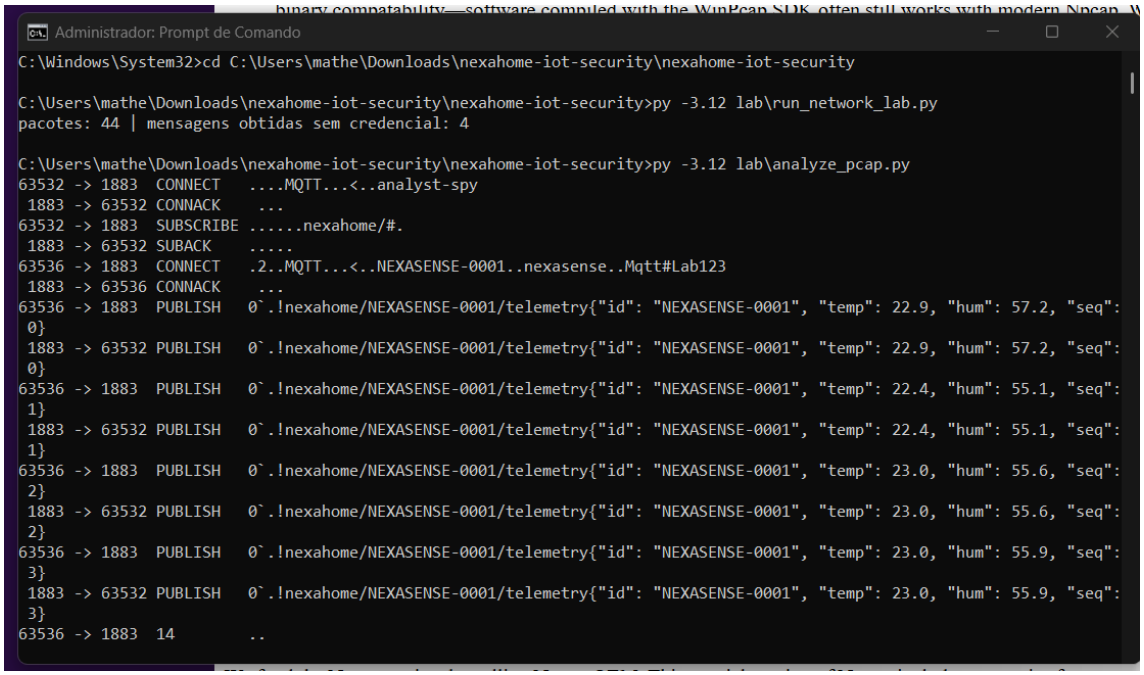


Figura – Execução no notebook Windows do analista (01/11/2026, Npcap 1.90, Python 3.12): 44 pacotes capturados; o CONNECT do NEXASENSE-0001 expõe usuário e senha em claro; 4 mensagens lidas por assinante sem credenciais.

8. Validação das mitigações (laboratório hardened)

Para provar que as recomendações funcionam, o mesmo laboratório foi repetido com TLS na porta 8883, login individual por dispositivo (hash Argon2) e ACL por tópico (lab/run_hardened_lab.py). Resultado: 7/7 testes aprovados, executados no notebook Windows do analista em 01/11/2026.

Teste	Resultado observado
T1 Cliente anônimo	Recusado (Not authorized)
T2 Senha incorreta	Recusado (Not authorized)
T3 MQTT em claro na porta TLS	Sem resposta (conexão falha)
T4 Dispositivo autenticado publica no próprio tópico	3 de 3 mensagens entregues ao analista autenticado
T5 Publicação em tópico de outro dispositivo	0 mensagens forjadas entregues (ACL bloqueou)
T6 Captura na 8883	111 pacotes e 10.408 bytes de payload; senha, tópico e telemetria não aparecem em claro (antes: senha visível no CONNECT)
T7 Tráfego TLS	8 registros de handshake TLS identificados

Assinatura de firmware (F03): com Ed25519 (lab/firmware_signing_demo.py), a imagem original é aceita, e uma imagem com 1 bit alterado ou com o endereço do broker trocado é rejeitada no modo hardened e aceita no modo inseguro. Ressalva: o dispositivo é um modelo em Python do comportamento de verificação, não um ESP32 real; Secure Boot, Flash Encryption e JTAG off ficam como proposta conceitual.

9. Firmware real em ESP32 emulado (Tasmota32 + QEMU Espressif)

Como não havia firmware fornecido nem placa física, o laboratório foi complementado com um firmware real e público: a release oficial do Tasmota32 (tasmota32.factory.bin, imagem completa de flash de 3.152.416 bytes, SHA-256 a0c6ed5fa59e18885db435cb22cb0456d6c9bce92477092462224ab6fa5942eb, obtida em 01/11/2026 de github.com/arendst/Tasmota), executado no emulador QEMU oficial da Espressif (máquina esp32). Não é uma placa física: o QEMU emula CPU e flash, mas não o Wi-Fi. Apenas arquivos públicos foram usados; nenhum dispositivo de terceiros foi atacado.

Etapa	Evidência real obtida
Etapa 2 – preservação	Original somente leitura + hash SHA-256 + cópia de trabalho (firmware/real/flash-hashes.txt)
Etapa 3 – estrutura	Bootloader (magic 0xE9) e tabela de partições reais: nvs, otadata, safeboot, app0 (0xE0000), spiffs; app ESP-IDF 5.5.5 com 6 segmentos; entropia 6,98
Etapa 3 – assinatura	36 bytes após os segmentos: sem bloco de assinatura Secure Boot v2 (hash ≠ autenticação)

Etapa	Evidência real obtida
Etapa 3 – strings	URLs de OTA em HTTP; campos Password, WebPassword, MqttPassword; mensagem “'Webpassword' is recommended”
Conferência cruzada (VM Kali)	sha256sum idêntico ao do relatório; binwalk confirma o bootloader, as 3 imagens ESP (safeboot e app0) e as 5 partições; entropia 6,984 igual à do script; execução no QEMU reproduziu o mesmo log de boot (firmware/real/kali_crosscheck.txt)
Etapa 4 – UART (real, emulada)	Log de boot do ROM e do bootloader seguido de “Tasmota Version 16.0.0 … ESP32-D0WDQ6”, “CFG: Use defaults”, “WifiManager active for 3 minutes”

Observação: o binwalk mostra “flags: 0x0 (not encrypted)” nas entradas da tabela de partições; isso é o flag da própria tabela e não informa o estado da Flash Encryption nas eFuses. Também aparecem caminhos de build (/home/runner/.platformio/…) e certificados X.509 públicos embutidos, sem relevância de risco aqui. Limitações dessa parte: o Guru Meditation Error ao iniciar o Wi-Fi é limitação do emulador (acesso a periférico não emulado), não vulnerabilidade; o estado real das eFuses (Secure Boot, Flash Encryption, JTAG) não é avaliável sem hardware; OTA e MQTT do Tasmota não foram testados dinamicamente. Leitura correta: isto não prova que o Tasmota seja inseguro, e sim que uma build padrão do ESP-IDF não vem assinada nem cifrada e que essas proteções precisam ser habilitadas pelo fabricante do produto. Os achados R01–R04 abaixo têm evidência real; F01–F07 vêm do cenário didático simulado.

Reprodução: lab/run_qemu_tasmota.sh (Linux, WSL ou VM Kali) e python lab/analyze_firmware.py firmware/real/tasmota32_factory_flash.bin.

10. Riscos identificados e classificação

10.1 Achados com evidência real (R01–R04)

Achado	Evidência	Impacto	Prob.	Risco	Recomendação
R01 Imagem de aplicação sem assinatura (sem bloco Secure Boot v2)	firmware/real/real-analysis-output.txt: 36 bytes após os segmentos (padding+hash); imagem assinada teria +4096 bytes	Alto	Baixa	Médio	Habilitar Secure Boot v2 e assinar as imagens no build de produção
R02 Flash não cifrada: conteúdo legível na imagem	firmware/real/flash-analysis-output.txt: entropia 6,98 bits/byte; ~11 mil strings legíveis (URLs, nomes de parâmetros, formulários)	Médio	Média	Médio	Flash Encryption (release) e NVS cifrada. Observação: a imagem de fábrica analisada não contém segredos reais
R03 URLs de OTA padrão em HTTP (strings do firmware)	strings: http://ota.tasmota.com/tasmota32/release/tasmota32.bin e ...-safeboot.bin	Alto	Baixa	Médio (a confirmar)	OTA por HTTPS com validação de certificado e imagem assinada
R04 Boot na UART expõe versão exata, hardware e estado de configuração, sem autenticação	hardware/evidence/uart_boot_REAL_QEMU_tasmota.log: 'Tasmota Version 16.0.0 … (2026-10-16)', 'ESP32-D0WDQ6', 'CFG: Use defaults', 'WifiManager active for 3 minutes'	Baixo	Média	Baixo	Reduzir log em produção e proteger/remover os pads de UART

10.2 Achados do cenário didático NexaSense (F01–F07, dispositivo simulado)

Achado	Evidência	Impacto	Prob.	Risco	Recomendação
F01 Segredos em texto claro no firmware, NVS e partição config	firmware/analysis-output.txt: wifi.sta.pass, api.token, mqtt password e admin:admin legíveis com strings; entropia 0,007 bits/byte (imagem não cifrada)	Alto	Média	Alto	Flash Encryption; credenciais por dispositivo provisionadas na produção; nada de segredo no binário
F02 MQTT sem TLS e credenciais em claro no CONNECT	network/evidence/pcap_analysis.txt: CONNECT com usuário nexasense e senha Mqtt#Lab123 visíveis; telemetria JSON em claro	Alto	Média	Alto	MQTT sobre TLS (8883) com validação de certificado; mTLS por dispositivo
F03 Atualização OTA via HTTP, sem assinatura e sem validação de certificado	strings: 'ota: update url=http://...', 'signature verification SKIPPED', 'TLS cert verification disabled'; config verify_signature=false	Alto	Média	Alto	HTTPS com pinning + imagem assinada + Secure Boot v2 + rollback protegido

Achado	Evidência	Impacto	Prob.	Risco	Recomendação
F04 Broker aceita assinante anônimo e sem ACL (configuração do laboratório)	network/evidence/anonymous_subscriber.txt: 4 mensagens de telemetria lidas sem credencial	Médio	Alta	Alto	allow_anonymous=false; ACL por clientid; um tópico por dispositivo
F05 Secure Boot e Flash Encryption desativados; JTAG habilitado	bootloader: 'secure boot: disabled', 'flash encryption: disabled'; 'efuse: JTAG_DISABLE=0'	Alto	Baixa	Médio	Habilitar Secure Boot v2 + Flash Encryption (modo release) e desabilitar JTAG via eFuse na versão comercial
F06 UART0 com shell de diagnóstico e log de boot verboso	hardware/evidence/uart_boot_SIMULATED.log (simulado a partir de strings reais do firmware): senha MQTT impressa no boot, prompt de shell	Médio	Média	Médio	Desativar console/shell em produção, reduzir nível de log, remover pads ou protegê-los
F07 Página /debug HTTP com credencial padrão admin:admin (indício)	string 'web: http://192.168.56.20/debug admin:admin' no firmware. NÃO validado dinamicamente no laboratório	Alto	Média	Alto (a confirmar)	Remover a interface de produção; sem senhas padrão (credencial única por dispositivo)

Matriz: Alto×Média=Alto; Alto×Baixa=Médio; Médio×Alta=Alto; Médio×Média=Médio. F07 é indício estático, ainda não confirmado dinamicamente.

11. Relação com as referências

Referência	Aplicação aos achados
OWASP ISTG	Estrutura das etapas e cobertura das superfícies física, firmware e rede
NIST IR 8259A	Lacunas: proteção de dados (F01, F02), atualização (F03), controle de acesso a interfaces (F04–F06)
ETSI EN 303 645	5.1 senhas padrão (F01, F07); 5.3 atualizações (F03); 5.4 parâmetros sensíveis (F01); 5.5 comunicação segura (F02, F04); 5.6 superfície de ataque (F06, F07)
MITRE EMB3D	Extração de firmware por hardware e interfaces privilegiadas expostas (F01, F05, F06)
ESP-IDF Security	Secure Boot, Flash Encryption e desativação de JTAG (F03, F05)

12. Recomendações e proposta de hardening

Prio.	Medida	Achados
P0	MQTT sobre TLS com mTLS por dispositivo; credenciais únicas e sem segredos no binário; OTA por HTTPS com imagem assinada e anti-rollback	F01, F02, F03, F07
P1	Secure Boot v2 + Flash Encryption (release) + JTAG desabilitado por eFuse; ACL e remoção de acesso anônimo no broker	F04, F05
P2	Console/shell desativado, logs mínimos, remover /debug, pads protegidos, recuperação A/B de OTA	F06, F07
P3	Threat model EMB3D, revisão por release, política de divulgação e suporte de atualizações	todos

Medidas irreversíveis (eFuses) foram propostas apenas conceitualmente; exigem gestão de chaves e teste prévio em unidades de desenvolvimento.

13. Caso real pesquisado: Espressif ESP32, fault injection e Secure Boot (CVE-2019-15894)

Dispositivo: SoC ESP32 (o mesmo da NexaHome). Superfície: física; falhas de tensão/clock (fault injection). Problema: com glitches precisamente temporizados, um atacante com acesso físico pode contornar a verificação do Secure Boot na inicialização e executar código não verificado a partir da flash; sem Flash Encryption, isso permite execução arbitrária. Reportado por Riscure (Yashin Mehaboobe e Santiago Cordoba Pellicer) e, independentemente, por LimitedResults; aviso da Espressif de 03/09/2019. Mitigação: atualizar o ESP-IDF (3.0.9, 3.1.6, 3.2.3, 3.3.1 ou ≥ 4.0), que impede desativar a Flash Encryption quando o Secure Boot é usado, e adotar versões de chip com ROM reforçada. Lição para a NexaHome: Secure Boot não basta isoladamente; é preciso defesa em profundidade (Secure Boot + Flash Encryption + JTAG off) e plano de atualização, pois quem tem a placa em mãos é um atacante realista para sensores instalados em ambientes de clientes. Fonte:

Espressif Security Advisory Concerning Fault Injection and Secure Boot; NVD CVE-2019-15894.

14. Limitações

Sem placa física: dispositivo NexaSense e log UART didáticos simulados, firmware real apenas em emulador; verificação de assinatura modelada em Python; Secure Boot/Flash Encryption não gravados (irreversíveis); análise do Tasmota apenas estática; OTA e /debug sem teste dinâmico; broker de laboratório configurado de forma permissiva por design, então F04 descreve a configuração de teste e deve ser verificada no broker real do piloto. Referências: OWASP ISTG, NIST IR 8259A, ETSI EN 303 645 V3.1.3, MITRE EMB3D e documentação ESP-IDF Security.